Harvest Now, Decrypt Later Attacks on TLS and the Migration to ML-KEM

Dogăreci Bianca-Alexandra, Iscru Bianca

University of Bucharest, Faculty of Mathematics and Informatics

Abstract. The threat of "harvest now, decrypt later" (HN-DL) attacks fundamentally challenges current Transport Layer Security (TLS): an adversary can record encrypted TLS handshakes today and decrypt them once a sufficiently powerful quantum computer becomes available. This paper examines the vulnerability of TLS 1.3 to HN-DL attacks, presents ML-KEM (Module-Lattice-Based Key Encapsulation Mechanism), the NIST-standardized lattice-based key encapsulation mechanism, as a practical solution, and provides experimental performance measurements.

1 Introduction

The security of modern network communications relies heavily on public-key cryptography. TLS is the protocol that encrypts the majority of Internet traffic. Every TLS 1.3 session begins with a handshake in which client and server perform an ephemeral Diffie–Hellman key exchange (ECDHE) to establish a shared secret, from which all session keys are derived.

Although large-scale quantum computers do not yet exist, this handshake is already a target. An adversary with access to network traffic can record and store the encrypted communications. Once a cryptographically relevant quantum computer becomes available, attackers may be able to break the TLS handshake's key exchange and decrypt previously recorded traffic. This is known as the *harvest now, decrypt later* (HNDL) attack [2].

2 The Harvest-Now, Decrypt-Later Attack

Encrypted network traffic is usually only useful to an attacker if it can be decrypted within a relatively short timeframe. However, this assumption is weakened when considering future advances in large-scale quantum computing.

The HNDL attack proceeds in two temporally separated phases:

1. **Harvest phase:** A passive adversary only needs network access and storage to intercept and archive TLS handshakes and encrypted records. This phase is typically difficult to detect, as it does not modify packets or disrupt communication flows.
2. **Decrypt phase:** After a quantum computer capable of breaking the relevant cryptographic primitives becomes available, the adversary processes the archived handshakes to recover session keys and decrypt the recorded application data.

3 The HN-DL Threat to TLS

TLS consists of two phases: the handshake, which establishes a shared secret and authenticates the server using digital signatures (RSA or ECDSA-Elliptic Curve Digital Signature Algorithm), and the record protocol, which encrypts application data using symmetric AEAD(Authenticated Encryption with Associated Data) ciphers (AES-GCM or ChaCha20-Poly1305).

TLS 1.3's quantum exposure lies entirely in the handshake:

- **Key exchange (ECDHE):** The security of ECDHE relies on the hardness of the elliptic curve discrete logarithm problem. A quantum adversary could solve this problem using Shor's algorithm [7], recovering the shared secret.
- **Server authentication (RSA/ECDSA):** Both schemes are also broken by Shor's algorithm, enabling server impersonation [3].

3.1 Why Forward Secrecy Does Not Help

Forward secrecy in TLS 1.3 guarantees that no long-lived material is involved in deriving session keys, so compromising the server at a later point exposes nothing about past sessions. However, in an HN-DL attack, this means the adversary must break the ephemeral Diffie-Hellman key exchange for each targeted session independently, rather than relying on a single long-term key.

4 Migrating to Post-Quantum TLS with ML-KEM

4.1 ML-KEM: The NIST Standard

ML-KEM (formerly Kyber) is a Key Encapsulation Mechanism standardized by NIST in FIPS 203 (August 2024) [6]. It is based on the Module Learning With Errors (MLWE) problem, which remains resistant against quantum computers.

Hybrid ML-KEM deployment has already reached production scale: Google Chrome and Cloudflare were among the first to deploy a preliminary variant of Kyber in TLS 1.3 [5], and AWS has since extended support to its Application and Network Load Balancers [1], collectively demonstrating production readiness.

4.2 How ML-KEM Works

ML-KEM departs from the Diffie–Hellman model: rather than having both endpoints contribute to the secret, one party generates the shared secret and simply encrypts it to the other's public key. More precisely: Alice generates a key pair and sends her public key to Bob. Bob uses Alice's public key in an encapsulation function, which outputs a fresh shared secret and a ciphertext. Bob sends back only the ciphertext. Alice decapsulates the ciphertext using her private key to recover the same secret. Both parties derive the session key from this secret using HKDF (HMAC-based Key Derivation Function). An attacker who intercepts the public key and ciphertext cannot recover the secret without Alice's private key.

4.3 Hybrid Key Exchange

During the transition to post-quantum cryptography, TLS deployments commonly use a hybrid approach for compatibility. Hybrid key exchange combines a classical ECDH exchange (X25519, a modern elliptic-curve Diffie–Hellman function) with a post-quantum KEM (ML-KEM-768) in parallel. The two shared secrets are concatenated to produce the final session key. This approach provides defense in depth: even if one component is broken, the other remains secure.

5 Experimental Methodology and Results [4]

We measured only the cryptographic operations that differ between classical and post-quantum handshakes: key generation and shared secret derivation. Our measurements report total (client and server) CPU work. We implemented classical X25519 and ML-KEM-768 handshakes in Python. Each variant ran 100 iterations, with timing measured using `time.perf_counter()` (microsecond precision). X25519 benefits from parallelism in real deployments, halving its latency. ML-KEM has sequential dependencies, so its latency matches our measurements.

All experiments ran on a system with an Intel Core processor (Intel64 Family 6 Model 165), 16 GB RAM, and Windows 11 (build 26200).

Table 1 compares handshake sizes. ML-KEM-768 requires 2,272 bytes on the wire, a 35.5× increase over X25519's 64 bytes.

Table 1. Handshake message exchange

Message	**X25519**	**ML-KEM-768**
Client → Server	Public key (32 bytes)	Ciphertext (1,088 bytes)
Server → Client	Public key (32 bytes)	Public key (1,184 bytes)
Total	**64 bytes**	**2,272 bytes**
Increase factor	1.0×	35.5×

Table 2 presents timing measurements. ML-KEM-768 requires 448 μs (0.448 ms) per handshake, a 3.1× slowdown compared to X25519's 144 μs (0.144 ms). ML-KEM decapsulation (254 ± 46 μs) is the most expensive and most variable operation, while X25519 operations show consistent performance with lower standard deviation.

Table 2. Computational overhead (microseconds, mean ± standard deviation)

Operation	**X25519**	**ML-KEM-768**
Key generation	82 ± 14	100 ± 22
Shared secret derivation	62 ± 3	348 ± 66
Total handshake	144 ± 17	448 ± 88
Slowdown factor	1.0×	3.1× (lower bound)

5.1 Network Security Implications

The 35.5× wire size increase means ML-KEM handshakes exchange 2,272 total bytes across two messages (1,184 bytes + 1,088 bytes), each below the Ethernet Maximum Transmission Unit (MTU) of 1,500 bytes. No fragmentation is required. The 3.1× computational slowdown (0.448 ms vs 0.144 ms) remains acceptable for general server infrastructure, but increases Denial of Service (DoS) attack surface by a factor of 3.1 and may cause handshake timeouts on low-power Internet of Things (IoT) devices.

6 Conclusion

The harvest now, decrypt later attack makes the quantum threat an immediate concern. ML-KEM (FIPS 203) provides a practical solution by replacing classical key exchange with lattice-based cryptography. Our measurements show a 3.1× slowdown and 35.5× wire size increase, both acceptable for deployment. Hybrid X25519+ML-KEM-768 is already supported in production environments.

References

1. Amazon Web Services: AWS application and network load balancers now support post-quantum key exchange for TLS. AWS What's New (Nov 2025), https://aws.amazon.com/about-aws/whats-new/2025/11/network-load-balancers-post-quantum-key-exchange-tls/, accessed: 2026-02-23
2. Blanco-Romero, J., Mendoza, F.A., Rubio, C.G., Campo, C., Sánchez, D.D.: On the practical feasibility of harvest-now, decrypt-later attacks (2026), https://arxiv.org/abs/2603.01091
3. Chen, L., Jordan, S., Liu, Y.K., Moody, D., Peralta, R., Perlner, R., Smith-Tone, D.: Report on post-quantum cryptography. Tech. rep. (Apr 2016). https://doi.org/10.6028/NIST.IR.8105
4. Dogăreci, B.A., Iscru, B.: Hndl-attack-tls-migration-to-ml-kem. https://github.com/BiancaDogareci/HNDL-attack-TLS-migration-to-ML-KEM (2025), gitHub repository containing experiment code and paper
5. Mallick, T., Kundu, A., Kompella, R.: Study of post quantum status of widely used protocols (2026), https://arxiv.org/abs/2603.28728
6. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM). Federal Information Processing Standard FIPS 203, NIST (2024). https://doi.org/10.6028/NIST.FIPS.203
7. Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. In: Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS). pp. 124–134. IEEE (1994). https://doi.org/10.1109/SFCS.1994.365700